

Игровой ИИ NPC

Прототип на Unity (C#): игрок и два типа вражеских NPC — «Страж» с полем зрения 180° и Raycast-видимостью и телепортирующийся «Призрак» со сканом 360°. Поведение реализовано конечными автоматами Idle · Patrol · Chase · Attack · Search · Teleport.

Unity 6000.0.70f1 · URP 2D · C# · сентябрь 2026

Содержание

1. Цель работы	2
2. Постановка задачи	2
3. Теоретическая справка	2
4. Схемы конечных автоматов	3
4.1. Общая архитектура классов	3
4.2. Автомат «Стража» (обычный противник)	3
4.3. Автомат «Призрака» (телепортирующийся противник)	4
5. Ключевые механики	6
5.1. Движение (не менее 8 направлений)	6
5.2. Поле зрения 180° и Raycast (Страж)	6
5.3. Скан 360° (Призрак)	7
5.4. Телепортация и её ограничения	7
5.5. Атака и обратная связь	8
5.6. Память и поиск	8
6. Параметры NPC	9
7. Реализация в Unity	9
7.1. Демонстрация работы (скриншоты из Play Mode)	9
8. Контрольные вопросы	10
9. Заключение	12
Приложение. Перечень файлов исходного кода	12

1. Цель работы

Изучить принципы построения искусственного интеллекта игровых персонажей на основе конечного автомата состояний (Finite State Machine, FSM) и реализовать базовое поведение вражеских NPC в игровом прототипе на движке **Unity (C#)**:

- выделять состояния игрового персонажа и условия переходов между ними;
- реализовывать обнаружение игрока (поле зрения 180° + Raycast, скан 360°);
- организовывать преследование и атаку;
- учитывать особенности восприятия NPC (память, потеря цели, поиск);
- проектировать различное поведение двух типов врагов, включая телепортирующегося NPC.

2. Постановка задачи

Разработать игровой прототип с персонажем игрока и двумя типами вражеских NPC, принимающих решения самостоятельно — в зависимости от положения игрока и состояния мира:

Тип NPC	Обнаружение	Особенность перемещения
«Страж» (обычный)	направленное поле зрения 180° , дальность 5,5 м, проверка препятствия Raycast (Line of Sight)	движение в 8 направлениях , патруль по точкам
«Призрак» (телепортирующийся)	периодический скан 360° : радиус 1 м, каждые 0,5 с + Line of Sight	свободное блуждание + телепортация (состояние TELEPORT)

Каждый враг реализован конечным автоматом: минимально **Idle** → **Patrol** → **Chase** → **Attack** и **Chase** → **Search** → **Patrol**; у Призрака автомат дополнен состоянием **Teleport**.

3. Теоретическая справка

Конечный автомат состояний (FSM) — модель поведения, в которой объект в каждый момент времени находится ровно в **одном** из конечного множества состояний. Смена состояния происходит по **переходам** — правилам вида «если в состоянии А выполнилось условие U — перейти в состояние В». Пока условие не выполнено, автомат остаётся в текущем состоянии и каждый кадр выполняет его логику (движение, проверка условий). Такая модель наглядна, дешёва по производительности и легко отлаживается (текущее состояние видно в HUD).

В прототипе автомат реализован без привязки к конкретному врагу: интерфейс **IState** (методы Enter/Tick/Exit) + класс **StateMachine** (хранит текущее состояние, оповещает о смене). Конкретные враги (GuardEnemy, GhostEnemy) создают собственные наборы состояний — поэтому добавление третьего типа врага не требует менять ядро (см. вопрос 10 в разделе 8).

4. Схемы конечных автоматов

4.1. Общая архитектура классов

Структура классов прототипа (Unity, C#)

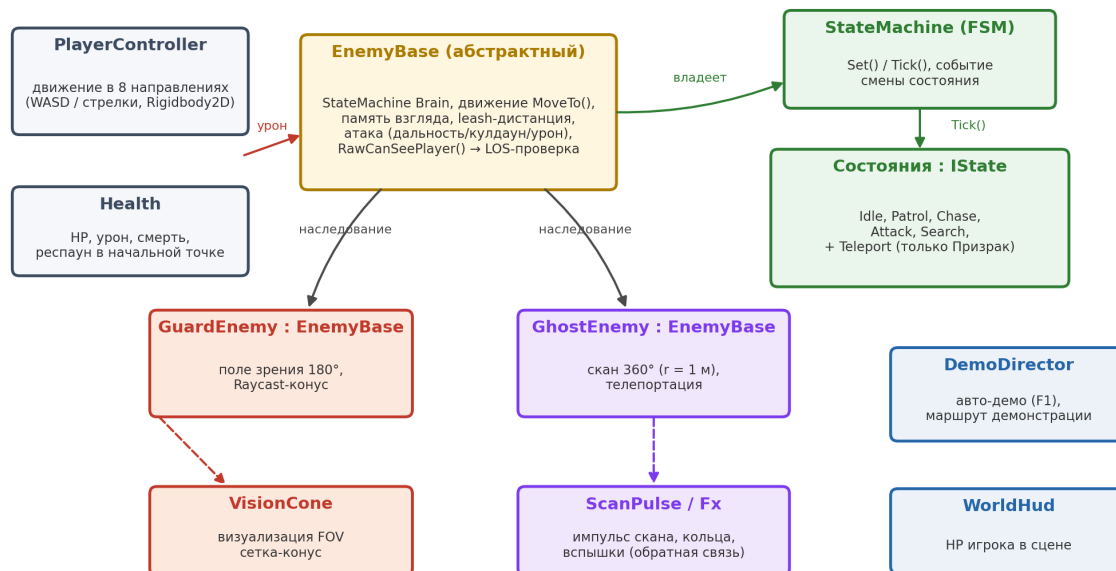


Рис. 1. Структура классов прототипа: ядро FSM и два типа врагов-наследника

EnemyBase — абстрактный базовый класс: владеет автоматом, движением, «памятью взгляда», атакой и общей проверкой Line of Sight. Наследники переопределяют только способ обнаружения (RawCanSeePlayer) и специфические механики (телепорт).

4.2. Автомат «Стража» (обычный противник)



Рис. 2. Граф конечного автомата «Стража»: Idle → Patrol → Chase → Attack и Chase → Search → Patrol

Текущее состояние	Условие	Следующее состояние
—	запуск сцены	Idle
Idle	истёк таймер ожидания (1,5 с)	Patrol
Idle	игрок обнаружен	Chase
Patrol	игрок обнаружен	Chase
Chase	дистанция ≤ радиуса атаки (0,95 м)	Attack
Chase	игрок потерян (не виден 0,4 с или убежал за 5,8 м)	Search
Attack	игрок вышел из радиуса атаки	Chase
Attack	игрок потерян	Search
Search	игрок обнаружен	Chase
Search	время поиска истекло (3 с осмотра)	Patrol

Таблица 1. Переходы автомата «Стража»

«Игрок обнаружен» = одновременно: **в пределах дальности (5,5 м) И внутри угла 180° И не закрыт препятствием** (Raycast). Одного расстояния недостаточно — за стеной или за спиной NPC игрок невидим.

4.3. Автомат «Призрака» (телепортирующийся противник)

Конечный автомат «ПРИЗРАК» — телепортирующий противник

обнаружение: периодический скан 360° (радиус 1 м, каждые 0,5 с) + Line of Sight; доп. состояние TELEPORT

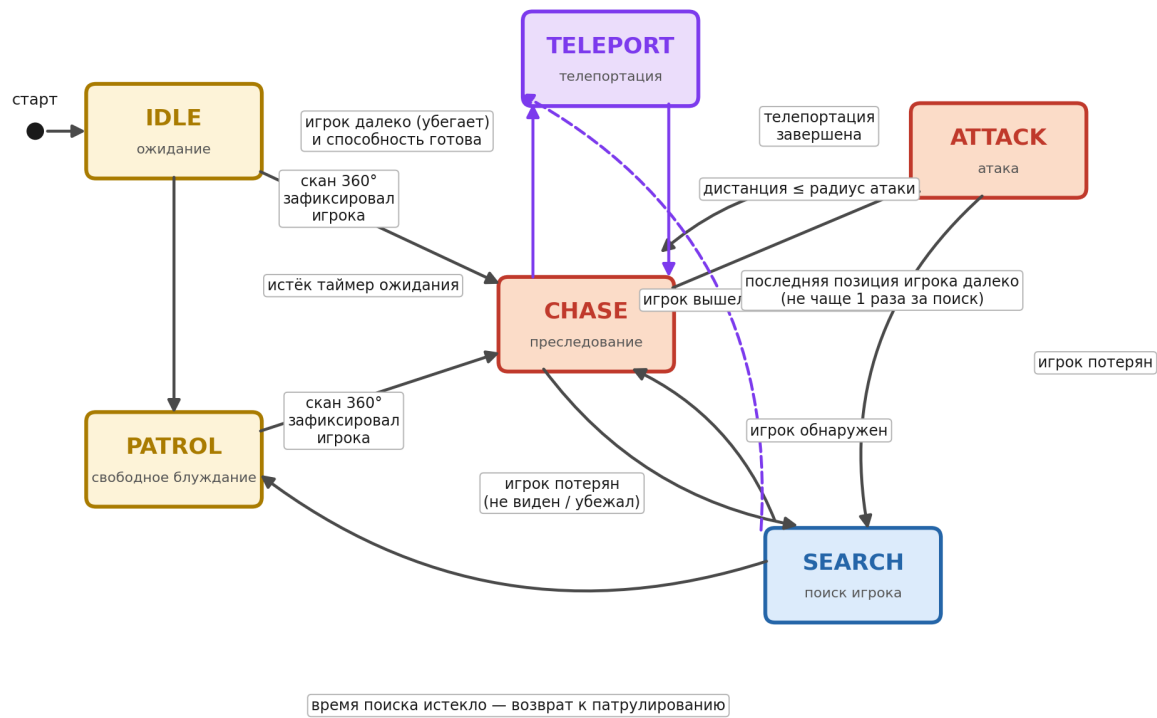


Рис. 3. Граф автомата «Призрака»: базовые состояния + TELEPORT (два триггера)

Текущее состояние	Условие	Следующее состояние
—	запуск сцены	Idle
Idle	истёк таймер ожидания	Patrol
Idle / Patrol	скан 360° зафиксировал игрока ($r \leq 1$ м, LOS)	Chase
Chase	дистанция $\leq$ радиуса атаки	Attack
Chase	игрок потерян (не виден / убежал за 4,5 м)	Search
Chase	игрок далеко ($> 3,2$ м — убегает) и телепорт готов	Teleport
Attack	игрок вышел из радиуса атаки	Chase
Attack	игрок потерян	Search
Search	игрок обнаружен	Chase
Search	последняя позиция далеко ($> 3,2$ м), не чаще 1 раза за поиск	Teleport
Teleport	телепортация завершена (~0,5 с)	Chase
Search	время поиска истекло	Patrol

Таблица 2. Переходы автомата «Призрака» (дополнения выделены)

Обоснование условия телепортации. Радиус обнаружения Призрака по заданию мал (1 м), а скорость (2,6 м/с) ниже скорости игрока (3,5 м/с): без способности враг никогда бы не догнал убегающую цель.

Поэтому основной триггер телепортации выбран в **Chase**: игрок оторвался более чем на 3,2 м (пытается убежать) и перезарядка (5 с) завершилась — Призрак совершает скачок к игроку и продолжает преследование/атаку. Второй триггер — в **Search**: если последняя известная позиция игрока далеко, Призрак не тратит время на пешее возвращение, а телепортируется к ней (не чаще одного раза за поиск, чтобы преследование не стало бесконечным и NPC в итоге вернулся к патрулированию).

5. Ключевые механики

5.1. Движение (не менее 8 направлений)

Перемещение и игрока, и NPC векторное ($\text{Rigidbody2D.velocity} = \text{direction} * \text{speed}$), поэтому доступны все направления, **включая 8 базовых** (вверх, вниз, влево, вправо и 4 диагонали); у игрока диагональ нормализуется — скорость по всем направлениям одинакова. Направление движения NPC каждый кадр пересчитывается по текущему положению цели, поворот «взгляда» выполняется плавно (интерполяция угла).

5.2. Поле зрения 180° и Raycast (Страж)

Assets/Scripts/Enemies/GuardEnemy.cs · EnemyBase.cs

```
// GuardEnemy.cs — три условия обнаружения
protected override bool RawCanSeePlayer()
{
    Vector2 to = (Vector2)player.position - (Vector2)transform.position;
    if (to.magnitude > viewDistance) return false;           // 1) дальность
    if (Vector2.Angle(facing, to) > viewAngleDeg * 0.5f) return false; // 2) угол 180°
    return HasLineOfSight();                                 // 3) нет стены
}

// EnemyBase.cs — проверка прямой видимости (Line of Sight)
protected bool HasLineOfSight()
{
    var hit = Physics2D.Linecast(transform.position, player.position, obstacleMask);
    return hit.collider == null; // между NPC и игроком не должно быть стены
}
```

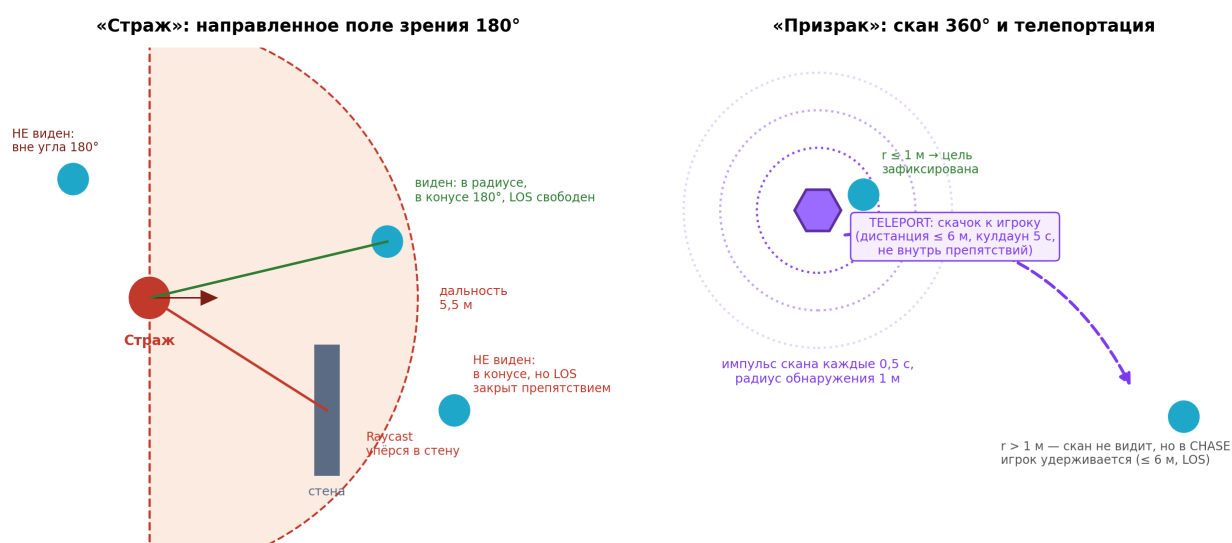


Рис. 4. Механика обнаружения: конус 180° + Raycast у Стража (слева); скан 360° и телепортация Призрака (справа)

Визуализация поля зрения — полупрозрачный «конус» из 60 лучей с обрезкой по стенам (компонент **VisionCone**); цвет конуса отражает состояние: зелёный — патруль, жёлтый — поиск, красный — погоня/атака. Луч трассировки и конус обрезаются препятствием — игрок, стоящий за стеной, не обнаруживается, даже если он в радиусе и в угле обзора.

5.3. Скан 360° (Призрак)

Assets/Scripts/Enemies/GhostEnemy.cs

```
// GhostEnemy.cs – периодический круговой скан + удержание цели
protected override bool RawCanSeePlayer()
{
    scanTimer += Time.deltaTime;
    if (scanTimer >= scanInterval)           // каждые 0,5 с
    {
        scanTimer = 0f;
        if (pulse != null) pulse.Play();      // визуальный импульс-кольцо
        if (DistanceToPlayer <= scanRadius && HasLineOfSight())
            PlayerLocked = true;              // цель зафиксирована (r = 1 м)
    }
    if (PlayerLocked && DistanceToPlayer <= trackDistance && HasLineOfSight())
        return true;                          // удержание: ≤ 6 м + LOS
    return false;
}
```

Если цель потеряна, Призрак не забывает её мгновенно: включается поиск по последней известной позиции (линия-«захват» цели отображается, пока игрок отслеживается).

5.4. Телепортация и её ограничения

- **перезарядка** — 5 с между скачками;
- **максимальная дистанция** — 6 м (желаемая точка клампится к радиусу);
- **точка назначения** — около цели, чуть дальше радиуса атаки;
- **запрет телепортации внутрь препятствий** — кандидат проверяется `Physics2D.OverlapCircle` по слою стен; если точка занята, перебираются соседние кандидаты кольцами (8 направлений × 3 радиуса), сортированные по близости к желаемой;
- **визуальная обратная связь** — фиолетовые расходящиеся кольца в точках ухода и прибытия; враг на ~0,3 с «растворяется» (спрайт и коллайдер отключаются).

Assets/Scripts/Enemies/GhostEnemy.cs

```
// GhostEnemy.cs – подбор безопасной точки телепортации
public bool FindTeleportDestination(Vector2 target, out Vector2 dest)
{
    Vector2 from = transform.position;
    Vector2 dir = target - from; float d = dir.magnitude;
    Vector2 desired = d > 0.01f ? target - dir / d * (attackRange * 0.75f) : target;
    Vector2 clamped = from + Vector2.ClampMagnitude(desired - from, maxTeleportDistance);

    var cands = new List<Vector2> { clamped }; // точка + кольца кандидатов
    float[] rs = { 0.7f, 1.4f, 2.1f };
    for (int ri = 0; ri < rs.Length; ri++)
        for (int i = 0; i < 8; i++)
        {
            float a = i * Mathf.PI / 4f + ri * 0.35f;
            cands.Add(clamped + new Vector2(Mathf.Cos(a), Mathf.Sin(a)) * rs[ri]);
        }
    cands.Sort((a, b) => (a - clamped).sqrMagnitude.CompareTo((b - clamped).sqrMagnitude));

    foreach (var c in cands)
        if (!Physics2D.OverlapCircle(c, 0.45f, obstacleMask)) // не внутри препятствия
        { dest = c; return true; }
    dest = default; return false; // безопасной точки нет – телепорт отменяется
}
```

5.5. Атака и обратная связь

Атака ближнего боя определена **дальностью** (0,95 м), **интервалом** (кулдаун 1,0 / 1,1 с) и **уроном** (12 / 8). Обратная связь: вспышка спрайта NPC, красное кольцо на игроке, уменьшение HP в HUD; при HP = 0 игрок исчезает и возрождается в начальной точке через 1,5 с.

5.6. Память и поиск

После потери видимости NPC не переключается мгновенно: краткая «память взгляда» (0,35–0,4 с) сглаживает мигание на границе конуса. Затем включается **Search**: NPC идёт к последней известной позиции игрока, осматривается (качание взгляда $\pm 110^\circ$) и ждёт 3 с; повторное обнаружение возвращает в Chase, иначе — Patrol. Дополнительно действует «поводок» (leash): если игрок оторвался дальше 5,8 м (Страж) / 4,5 м (Призрак), NPC прекращает бесконечную погоню и переходит к поиску.

Assets/Scripts/Enemies/States/ChaseState.cs

```
// ChaseState.cs – приоритет переходов в преследовании
public override void Tick()
{
    if (enemy.HasTeleportAbility && enemy.ShouldTeleport()) { enemy.GoToTeleport(); return; }
    if (!enemy.PlayerVisible || enemy.DistanceToPlayer > enemy.chaseLeashRange)
    { enemy.GoToSearch(); return; } // потерян / убежал
    float d = enemy.DistanceToPlayer;
    if (d <= enemy.attackRange) { enemy.GoToAttack(); return; }
    enemy.MoveTo(enemy.player.position); // направление каждый кадр
}
```

6. Параметры NPC

Параметр	Игрок	Страж	Призрак
Скорость, м/с	3,5	2,3	2,6
НР / урон / кулдаун атаки	100	12 / 1,0 с	8 / 1,1 с
Дальность атаки	—	0,95 м	0,95 м
Обнаружение	—	FOV 180°, 5,5 м, Raycast	скан 360°, r = 1 м, 0,5 с
Удержание цели	—	пока виден (память 0,4 с)	≤ 6 м + LOS (после фиксации)
Патруль	—	4 точки-маршрута	блуждание r ≤ 4,5 м
Поиск	—	3 с осмотра	3 с осмотра
Поводок погони	—	5,8 м	4,5 м
Телепорт	—	—	кулдаун 5 с; ≤ 6 м; > 3,2 м; не в стены

Таблица 3. Сводные параметры прототипа

7. Реализация в Unity

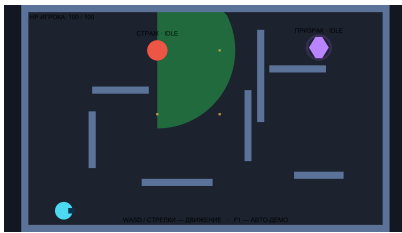
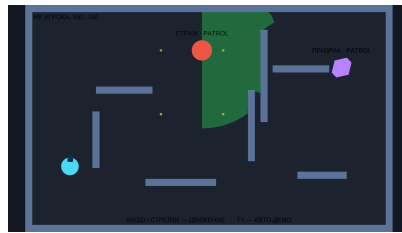
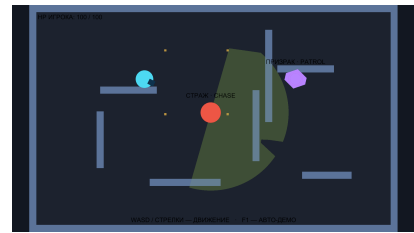
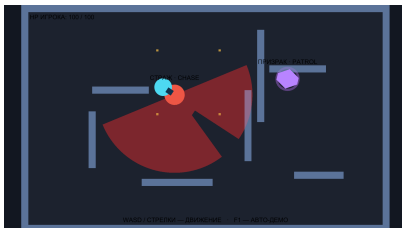
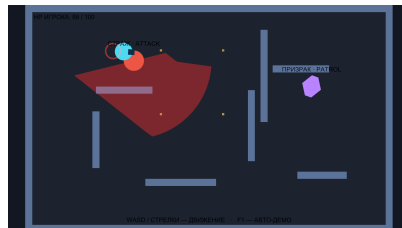
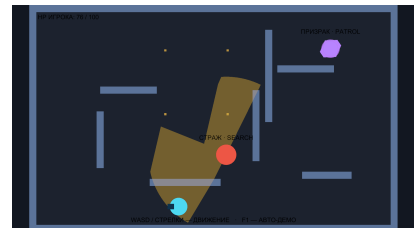
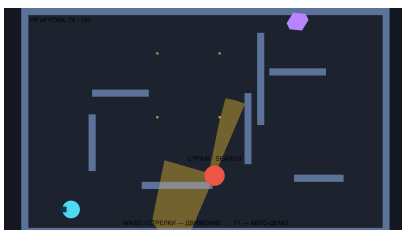
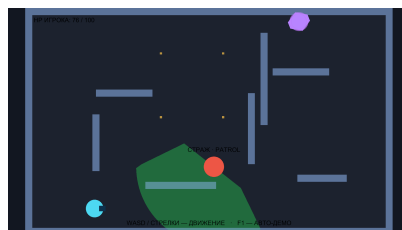
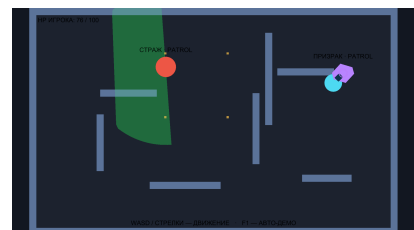
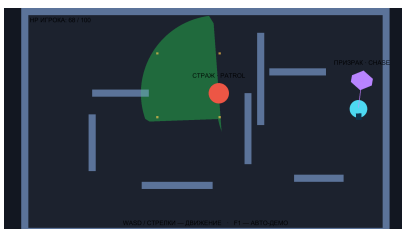
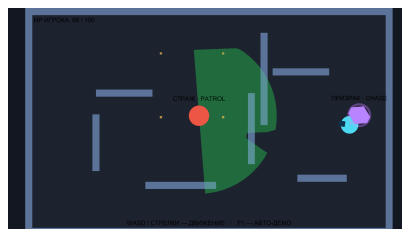
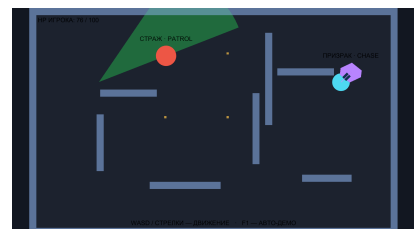
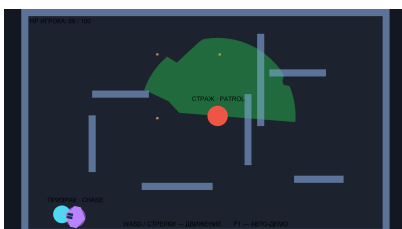
Проект: D:\Rabota\Unity\saves\AI_lab (Unity **6000.0.70f1**, URP 2D). Сборка сцены — одной командой меню: **AI Lab 1** → **Построить сцену лаборатории** (редактор-скрипт LabSceneBuilder создаёт уровень со стенами, игрока, обоих NPC, HUD и сохраняет Assets/Scenes/LabScene.unity; сборка занимает пару секунд).

Папка	Назначение
Scripts/Core/	Fsm.cs (IState + StateMachine), SpriteFactory, Fx, Health
Scripts/Player/	PlayerController — движение в 8 направлениях
Scripts/Enemies/	EnemyBase, GuardEnemy, GhostEnemy, VisionCone, ScanPulse
Scripts/Enemies/States/	Idle, Patrol, Chase, Attack, Search, Teleport
Scripts/Demo/	DemoDirector — авто-демо (F1)
Scripts/UI/	WorldHud — НР игрока в сцене
Editor/	LabSceneBuilder — сборка сцены; PlayModeCapture — захват скриншотов

Таблица 4. Структура исходного кода (Assets/)

Каждый NPC подписан своим именем и текущим состоянием прямо в сцене («СТРАЖ · CHASE»), НР игрока — в левом верхнем углу, поэтому переключения автоматов видны и на скриншотах, и при записи видео.

7.1. Демонстрация работы (скриншоты из Play Mode)

*Старт: оба NPC в Idle**Патрулирование по точкам**Обнаружение игрока**Преследование**Атака: урон по HP**Поиск: к последней позиции**Поиск: осмотр области**Возврат к патрулированию**Скан 360° Призрака**Телепортация (эффекты)**Погоня после скачка**Оба автомата одновременно**Финал демо-сценария**Рис. 5. Хронология демо-сценария (автоматический режим F1): все состояния обоих NPC*

8. Контрольные вопросы

Вопрос 1. Что такое конечный автомат состояний (FSM)?

Модель поведения, в которой объект в каждый момент находится ровно в одном состоянии из конечного набора, а смена состояний происходит по переходам — правилам «условие → новое состояние». Каждое состояние инкапсулирует свою логику (что NPC делает сейчас), что делает поведение наглядным и предсказуемым.

Вопрос 2. Чем состояние отличается от перехода?

Состояние — режим поведения, в котором NPC находится и выполняет некоторую логику каждый кадр (например, Chase — бежать к игроку). Переход — событие/условие, переключающее автомат из одного состояния в другое (например, «дистанция $\leq$ радиуса атаки»: Chase $\rightarrow$ Attack). Состояние отвечает на вопрос «что делать», переход — «когда сменить деятельность».

Вопрос 3. Какие состояния необходимы для базового поведения врага?

Idle (ожидание), Patrol (патрулирование), Chase (преследование), Attack (атака) и Search (поиск после потери игрока). Для телепортирующегося врага добавляется Teleport.

Вопрос 4. Почему для определения видимости игрока недостаточно только расстояния?

Расстояние — лишь необходимое условие. Игрок может находиться в радиусе, но за стеной (линия взгляда перекрыта препятствием — нужен Raycast/Line of Sight), вне сектора обзора (например, за спиной NPC с FOV 180°) или в «мёртвой зоне» механики обнаружения. Без этих проверок NPC «видел» бы сквозь стены и вокруг себя, что разрушает стелс-геймплей.

Вопрос 5. Как реализовать поле зрения NPC?

Три проверки: (1) расстояние до игрока $\leq$ дальности обзора; (2) угол между направлением взгляда NPC и направлением на игрока $\leq$ половине угла обзора (Vector2.Angle); (3) Raycast от NPC к игроку не пересекает слой препятствий. Для наглядности поле зрения рисуется веером лучей с обрезкой по стенам (компонент VisionCone).

Вопрос 6. Для чего используется Raycast при обнаружении игрока?

Для проверки прямой видимости (Line of Sight): луч из позиции NPC к игроку, пущенный по слою препятствий, определяет, есть ли на пути стена. Если луч попал в препятствие раньше, чем «дошёл» до игрока, игрок считается скрытым, даже если он в радиусе и в угле обзора.

Вопрос 7. Чем поле зрения 180° отличается от кругового обнаружения 360°?

Поле 180° — направленное: NPC видит только перед собой, имеет «слепую зону» за спиной и может быть обойдён сзади — появляется тактика подкрадывания. Круговое обнаружение 360° направления не имеет — цель видна с любой стороны, но в нашей реализации лишь на малом радиусе 1 м, что компенсируется телепортацией. Первое создаёт «честную» стелс-механику, второе — механику «не подходи близко к аномалии».

Вопрос 8. Какие преимущества и недостатки имеет FSM при разработке игрового AI?

Преимущества: простота и наглядность; детерминированность; дешевизна (проверки только текущего состояния); лёгкая отладка; состояния независимы. Недостатки: число переходов растёт квадратично с числом состояний; сложно выражать параллельные поведения и приоритеты; большие автоматы становятся нечитаемыми; нет долгосрочной памяти о целях. Для сложного AI переходят к иерархическим FSM, деревьям поведения (Behaviour Trees), utility-системам или GOAP.

Вопрос 9. Что произойдёт, если у NPC одновременно выполнятся условия перехода в несколько состояний?

В классическом FSM переход определяется первым удовлетворённым условием в порядке проверки, поэтому результат зависит от приоритетов — при неправильном порядке поведение становится ошибочным. В нашей реализации порядок в `Tick()` задан по приоритету: сначала специфические механики (телепорт), затем потеря цели, затем атака/движение; переход выполняется только один за кадр (`return` после смены состояния), что исключает осцилляции.

Вопрос 10. Как расширить систему при появлении третьего типа врага?

Ядро (`IState`, `StateMachine`, `EnemyBase`) не привязано к типам врагов: нужно унаследовать новый класс от `EnemyBase`, переопределить `RawCanSeePlayer` (свой способ обнаружения) и `CreateStates` (собственный набор состояний — можно переиспользовать существующие и добавить новые, как `TeleportState` у Призрака). Новые состояния реализуют `IState` и работают с любым наследником `EnemyBase` через его публичный API (`MoveTo`, `GoToChase`, `TryAttack`, ...). Ни один существующий класс менять не требуется — принцип открытости/закрытости.

9. Заключение

Разработан игровой прототип на Unity, в котором поведение двух типов вражеских NPC реализовано конечными автоматами состояний: обычный противник «Страж» (поле зрения 180°, Raycast-видимость, патруль/погоня/атака/поиск) и телепортирующийся «Призрак» (периодический 360°-скан, фиксация и удержание цели, ограниченная телепортация, состояние `Teleport`). Схемы автоматов, таблицы переходов, исходный код и демонстрация (авто-режим F1 + скриншоты) полностью соответствуют заданию. Автоматическая сборка сцены одной командой меню и подписи состояний в HUD делают прототип удобным для показа и защиты лабораторной работы.

Приложение. Перечень файлов исходного кода

Файл	Назначение
Scripts/Core/Fsm.cs	интерфейс <code>IState</code> , класс <code>StateMachine</code> , базовое <code>FsmState</code>
Scripts/Core/SpriteFactory.cs	процедурные спрайты (круг, гексагон, кольцо)
Scripts/Core/Fx.cs	эффекты: кольца, вспышки
Scripts/Core/Health.cs	HP игрока, смерть/респаун
Scripts/Player/PlayerController.cs	движение игрока в 8 направлениях
Scripts/Enemies/EnemyBase.cs	базовый класс врага (FSM, движение, память, атака)
Scripts/Enemies/GuardEnemy.cs	«Страж»: FOV 180° + LOS
Scripts/Enemies/GhostEnemy.cs	«Призрак»: скан 360°, телепортация
Scripts/Enemies/VisionCone.cs	визуализация поля зрения
Scripts/Enemies/ScanPulse.cs	импульс сканирования

Файл	Назначение
Scripts/Enemies/States/*.cs	Idle / Patrol / Chase / Attack / Search / Teleport
Scripts/Demo/DemoDirector.cs	авто-демо для записи видео
Scripts/UI/WorldHud.cs	HP в сцене
Editor/LabSceneBuilder.cs	сборка сцены лаборатории
Editor/PlayModeCapture.cs	пакетный захват скриншотов

Таблица 5. Исходный код прототипа (полные листинги — в папке проекта Unity)